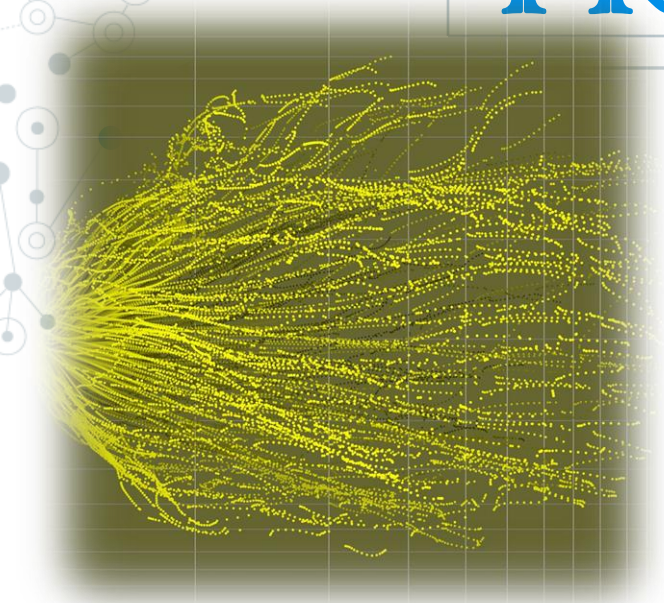
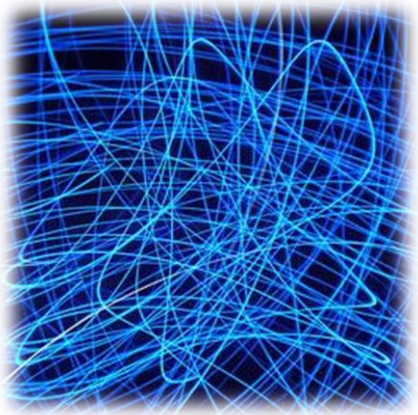
STOCHASTIC PROCESSES





DEFINITION

- A **stochastic process** is used to model systems that evolve in time and whose laws of evolution are probabilistic in nature.
  - The state of the system evolves in time and can be described through a **state variable** $x(t)$.
  - The evolution of the state of the system depends on the outcome of an experiment. We can write $x = x(t, \omega)$, where ω denotes the outcome of the experiment.
- Examples:
  - The random walk in one dimension.
  - Brownian motion.
  - The exchange rate between the British sterling and the US dollar.
  - Photon emission.
  - The spread of the SARS epidemic.

ONE-DIM RANDOM WALK

We let time be discrete, i.e. $t = 0, 1, \ldots$. Consider the following stochastic process S_n:

- $S_0 = 0$;
- at each time step it moves to ± 1 with equal probability $\frac{1}{2}$.

In other words, at each time step we flip a fair coin. If the outcome is heads, we move one unit to the right. If the outcome is tails, we move one unit to the left.
Alternatively, we can think of the random walk as a sum of independent random variables:

$$S_n = \sum_{j=1}^{n} X_j,$$

where $X_j \in \{-1, 1\}$ with $\mathbb{P}(X_j = \pm 1) = \frac{1}{2}$.

ONE-DIM RANDOM WALK

We can simulate the random walk on a computer:

- We need a **(pseudo)random number generator** to generate n independent random variables which are **uniformly distributed** in the interval [0,1].
- If the value of the random variable is $\geqslant \frac{1}{2}$ then the particle moves to the left, otherwise it moves to the right.
- We then take the sum of all these random moves.
- The sequence $\{S_n\}_{n=1}^{N}$ indexed by the discrete time $T = \{1, 2, \ldots N\}$ is the **path** of the random walk. We use a linear interpolation (i.e. connect the points $\{n, S_n\}$ by straight lines) to generate a **continuous path**.

ONE-DIM RANDOM WALK

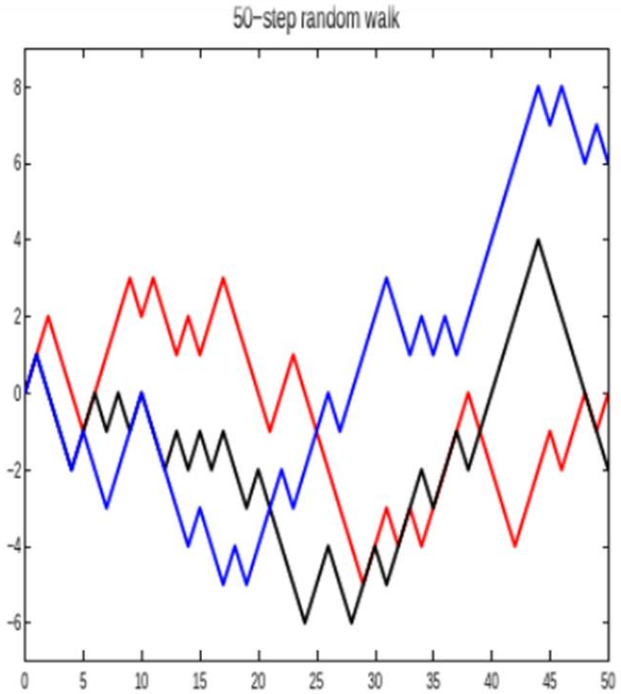


Figure: Three paths of the random walk of length $N = 50$.

ONE-DIM RANDOM WALK

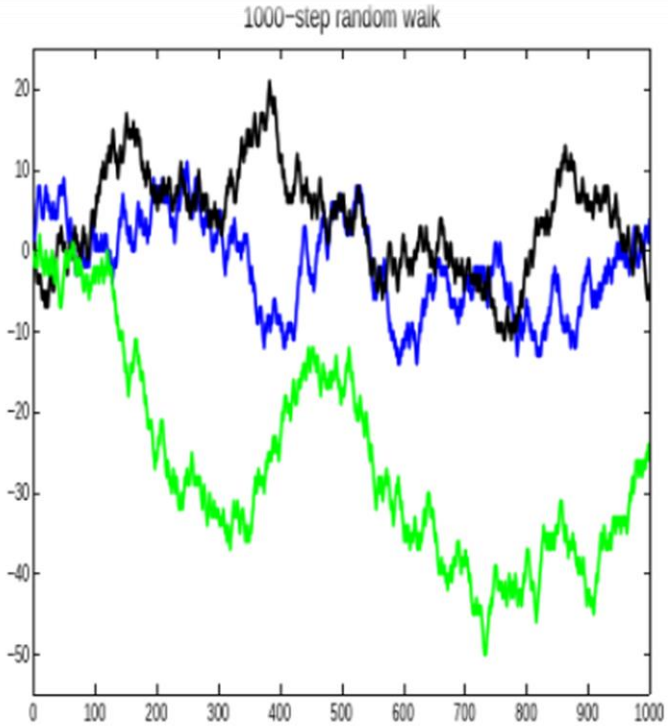


Figure: Three paths of the random walk of length $N = 1000$.

DISCRETE TIME SP

- Every path of the random walk is different: it depends on the outcome of a sequence of independent random experiments.
- We can compute statistics by generating a large number of paths and computing averages. For example, $\mathbb{E}(S_n) = 0$, $\mathbb{E}(S_n^2) = n$.
- The paths of the random walk (without the linear interpolation) are not continuous: the random walk has a jump of size 1 at each time step.

DISCRETE TIME SP

- This is an example of a **discrete time, discrete space** stochastic processes.
- The random walk is a **time-homogeneous** (the probabilistic law of evolution is independent of time) **Markov** (the future depends only on the present and not on the past) process.
- If we take a large number of steps, the random walk starts looking like a continuous time process with continuous paths.

CONTINUOUS TIME SP

- Consider the sequence of **continuous time** stochastic processes

$$Z_t^n := \frac{1}{\sqrt{n}} S_{nt}.$$

- In the limit as $n \to \infty$, the sequence $\{Z_t^n\}$ converges (in some appropriate sense) to a **Brownian motion** with **diffusion coefficient** $D = \frac{\Delta x^2}{2\Delta t} = \frac{1}{2}$.

BROWNIAN MOTION

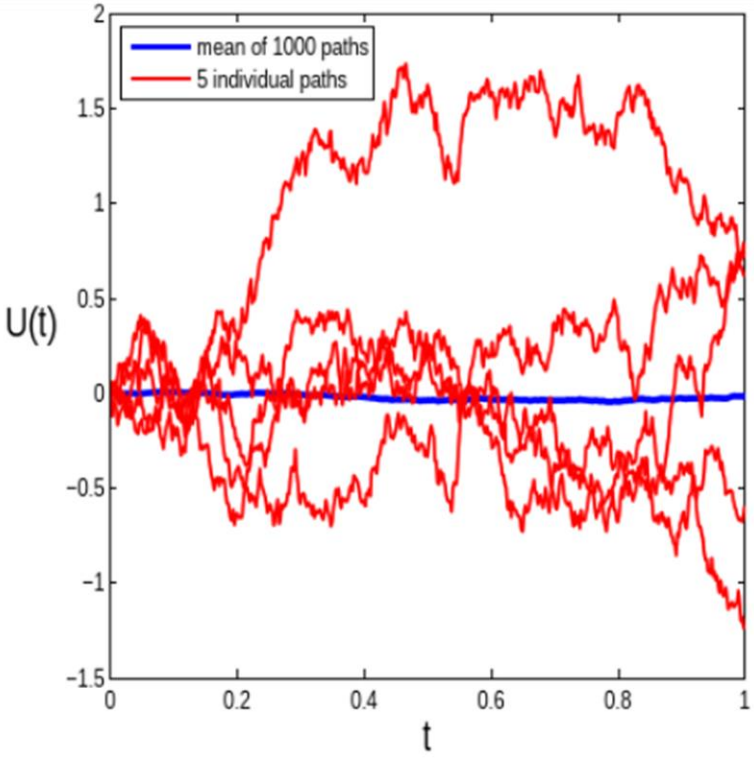
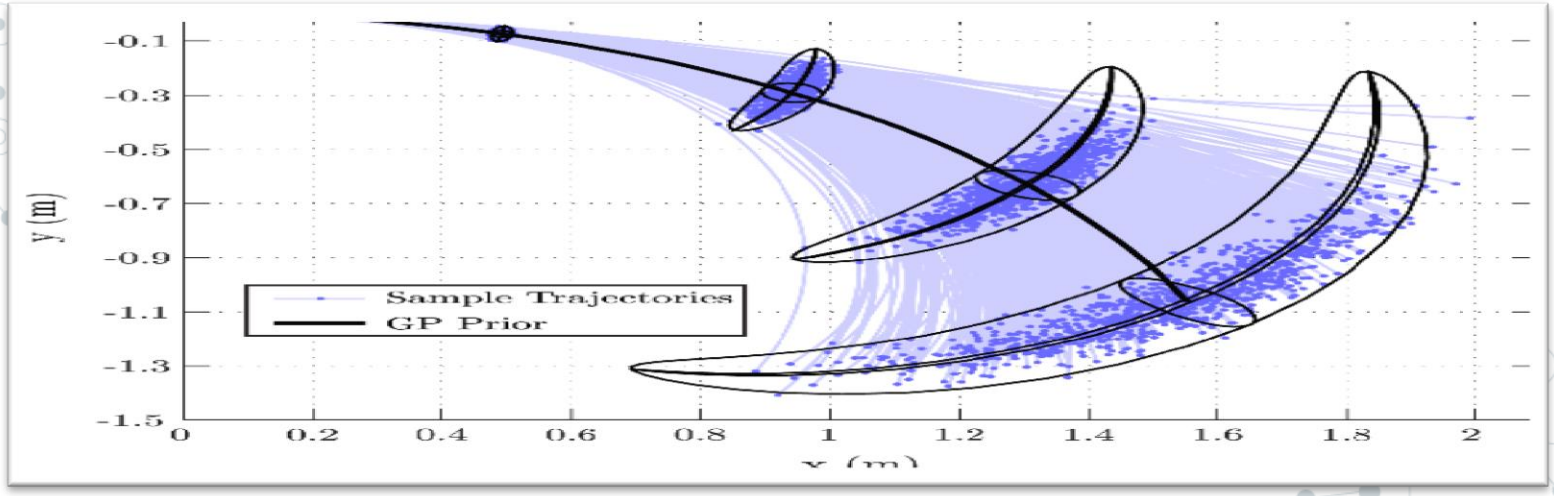


Figure: Sample Brownian paths.

GAUSSIAN PROCESSES



GAUSSIANS

The Gaussian distribution $\boldsymbol{X} \sim N(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ has the density function,

$$f_{\boldsymbol{X}}(\boldsymbol{x}) = \frac{1}{\sqrt{(2\pi)^p |\boldsymbol{\Sigma}|}} \exp\left\{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^T \boldsymbol{\Omega}(\boldsymbol{x}-\boldsymbol{\mu})\right\},$$

where $\boldsymbol{\Omega} = \boldsymbol{\Sigma}^{-1}$.

I will use **Ω** and **Λ** interchangeably!

CONDITIONAL DISTRIBUTION

Let $x \in \mathbb{R}^{n+p}$ be a Gaussian random vector which we will partition as

$$x = \begin{bmatrix} x_a \\ x_b \end{bmatrix},$$

where $x_a \in \mathbb{R}^n$ and $x_b \in \mathbb{R}^p$. Then the means and covariances can be represented as

$$\mu = \begin{bmatrix} \mu_a \\ \mu_b \end{bmatrix}, \qquad \Sigma = \begin{bmatrix} \Sigma_{aa} & \Sigma_{ab} \\ \Sigma_{ba} & \Sigma_{bb} \end{bmatrix}. \tag{11}$$

Here, Σ_{aa} is $n \times n$, Σ_{ab} is $n \times p$, Σ_{ba} is $p \times n$ and Σ_{bb} is $p \times p$.

MARGINAL DISTRIBUTION

Given $p(x_a, x_b)$ which is normally distributed with mean and covariance as in (11), the marginal distribution

$$p(x_a) = \int p(x_a, x_b)\, \mathrm{d}x_b$$

is also Gaussian with

$$\mathbb{E}[x_a] = \mu_a,$$
$$\mathrm{Cov}[x_a] = \Sigma_{aa}.$$

CONDITIONAL DISTRIBUTION

Given a partitioned matrix, the following formula holds

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix}^{-1} = \begin{bmatrix} M & -MBD^{-1} \\ -D^{-1}CM & D^{-1} + D^{-1}CMBD^{-1} \end{bmatrix},$$

where $M = (A - BD^{-1}C)^{-1}$.

using this formula and definition of the precision matrix, we can express the conditional mean and variance as

$$\mu_{a|b} = \mu_a + \Sigma_{ab}\Sigma_{bb}^{-1}(x_b - \mu_b),$$
$$\Sigma_{a|b} = \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}.$$

GAUSSIAN PROCESS

Definition

A stochastic process $\{X_t; t \in T\}$ is a Gaussian process if for any finite set of indices $\{t_1, \ldots, t_n\}$ of T, $(X_{t_1}, \ldots, X_{t_n})$ is a multivariate normal random variable.

GAUSSIAN PROCESS

If T is finite, the Gaussian process is a multivariate Gaussian distribution.

- If T is an infinite set, eg. a subset of $\mathbb{R}^d$, then this is an infinite-dimensional generalization of multivariate normal random variables.
- A Gaussian process is completely determined by its
  - Mean function $\mu(\cdot) : T \to \mathbb{R}$
  - Covariance function or kernel $k(\cdot, \cdot) : T \times T \to \mathbb{R}$
- The samples of a Gaussian process are paths if $T = \mathbb{R}$, or surfaces if $T = \mathbb{R}^d$, $d > 1$, and their smoothness is dependent on the kernel.

EXAMPLES OF KERNELS

- Radial basis function (RBF or "Gaussian"):

$$k(x,y) = e^{-\frac{\|x-y\|^2}{2\sigma^2}}, \quad x,y \in \mathbb{R}^d$$

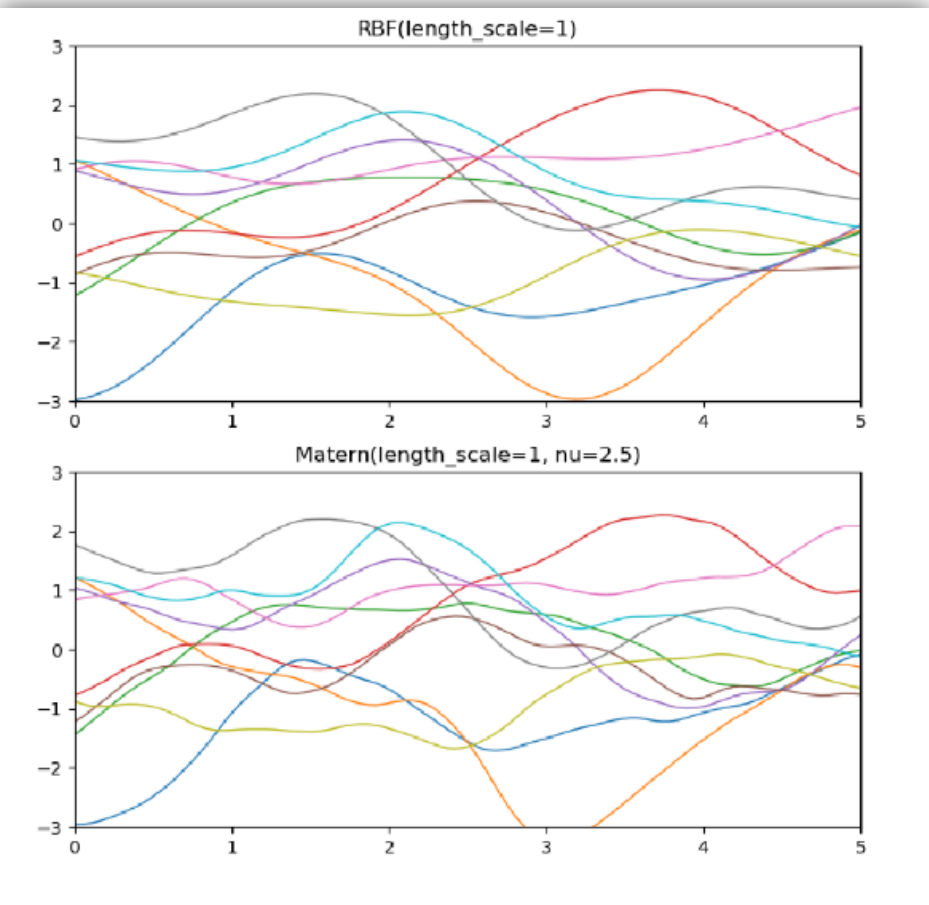
- Matérn $\frac{5}{2}$ ($\frac{5}{2} = \nu + \frac{1}{2}$, $\nu = 2$):

$$k(x,y) = \left(1 + \sqrt{5}\,\|x-y\| + \frac{5}{3}\,\|x-y\|^2\right) e^{-\sqrt{5}\|x-y\|}, \quad x,y \in \mathbb{R}^d$$

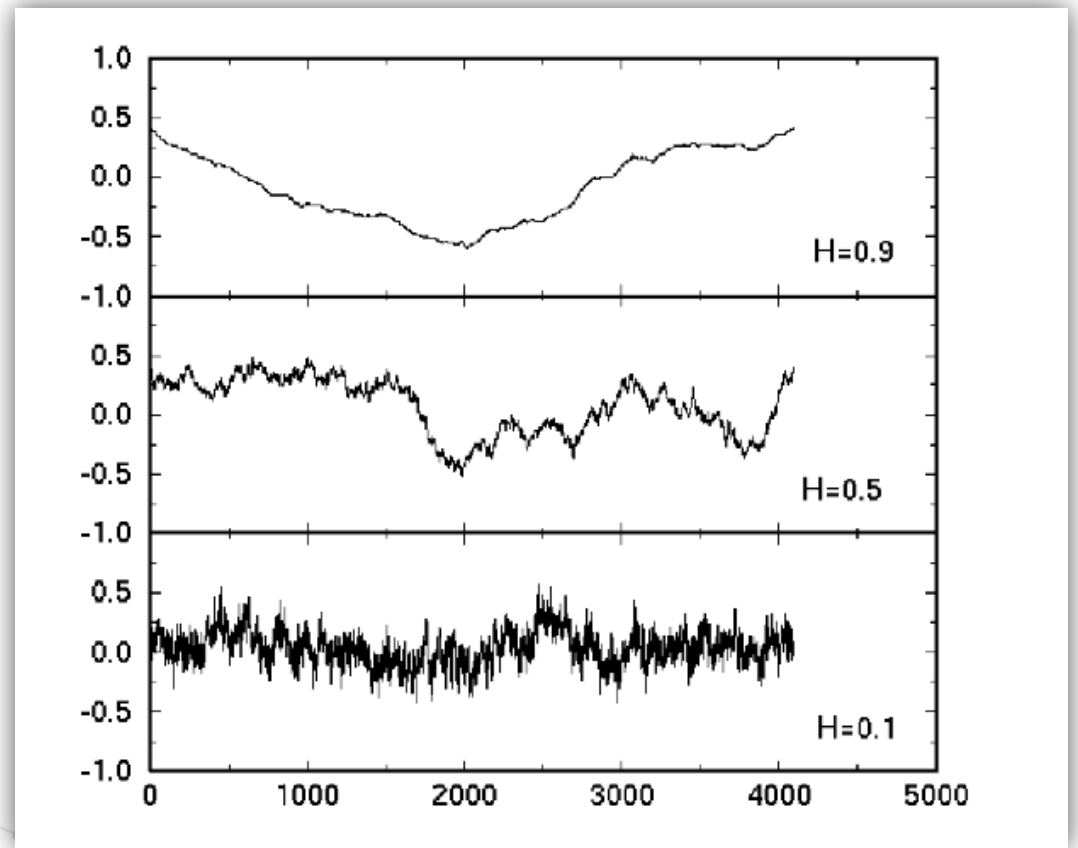
- Fractional Brownian motion, with Hurst parameter $H \in (0,1)$:

$$k(x,y) = \frac{1}{2}\left(x^{2H} + y^{2H} - |x-y|^{2H}\right), \quad x,y \in \mathbb{R}^+$$

EXAMPLES OF GP



FRACTAL BROWNIAN MOTION



GP REGRESSION

- Recall that we model

$$y = f(x) + \epsilon, \tag{15}$$

where we assume $\epsilon \sim \mathcal{N}(0, \tau^2)$ and is independent between samples.

- Previously, we have been assuming that the hypothesis function f is parametric, i.e. it can be described by a fixed number of parameters, e.g. $f(x) = \langle \theta, x \rangle$, which are to be learnt.

GP REGRESSION

- Returning to (15), we have

$$p(y|f(x)) \sim \mathcal{N}(f(x), \tau^2).$$

- Now instead of modeling f from a parameterized family of functions, we enforce a prior Gaussian distribution, with kernel K:

$$p(f(\cdot)) \sim \mathcal{N}(0, K).$$

GP REGRESSION

Conditioned on input values $x^{(1)}, \ldots, x^{(n)}$, we compute the marginal distribution of y by

$$p(y) = \int p(y|f)p(f)df.$$

We know this is normally distributed with mean

$$\mathbb{E}[y] = \mathbb{E}[f] + \mathbb{E}[\epsilon] = 0$$

and covariance C^n with entries

$$\begin{aligned} C_{ij}^n = \mathbb{E}\left[y^{(i)}y^{(j)}\right] &= \mathbb{E}\left[\left(f\left(x^{(i)}\right) + \epsilon^{(i)}\right)\left(f\left(x^{(j)}\right) + \epsilon^{(j)}\right)\right] \\ &= \mathbb{E}\left[f\left(x^{(i)}\right) f\left(x^{(j)}\right)\right] + \mathbb{E}\left[\epsilon^{(i)}\epsilon^{(j)}\right] \\ &= K(x^{(i)}, x^{(j)}) + \tau^2\delta_{ij}. \end{aligned} \tag{16}$$

GP REGRESSION

Conditioned on input values $x^{(1)}, \ldots, x^{(n)}$, we compute the marginal distribution of y by

$$p(y) = \int p(y|f)p(f)df.$$

We know this is normally distributed with mean

$$\mathbb{E}[y] = \mathbb{E}[f] + \mathbb{E}[\epsilon] = 0$$

and covariance C^n with entries

$$\begin{aligned} C_{ij}^n = \mathbb{E}\left[y^{(i)}y^{(j)}\right] &= \mathbb{E}\left[\left(f\left(x^{(i)}\right) + \epsilon^{(i)}\right)\left(f\left(x^{(j)}\right) + \epsilon^{(j)}\right)\right] \\ &= \mathbb{E}\left[f\left(x^{(i)}\right) f\left(x^{(j)}\right)\right] + \mathbb{E}\left[\epsilon^{(i)}\epsilon^{(j)}\right] \\ &= K(x^{(i)}, x^{(j)}) + \tau^2\delta_{ij}. \end{aligned} \tag{16}$$

GPR PREDICTION

- Given $\left(x^{(1)}, t^{(1)}\right), \ldots, \left(x^{(n)}, t^{(n)}\right)$, and a new input point $x^{(n+1)}$, we predict $y^{(n+1)}$ by computing the posterior distribution

$$p\left(y^{(n+1)} \,\middle|\, y^{(1)} = t^{(1)}, \ldots, y^{(n)} = t^{(n)}\right).$$

- The joint distribution over $y^{(1)}, \ldots, y^{(n)}, y^{(n+1)}$ has mean 0 and covariance C^{n+1} given by (16), which can be partitioned as

$$C^{n+1} = \begin{bmatrix} C^n & k \\ k^T & c \end{bmatrix},$$

where $k = \left[K\left(x^{(1)}, x^{(n+1)}\right), \ldots, K\left(x^{(n)}, x^{(n+1)}\right)\right]^T$ and $c = K\left(x^{(n+1)}, x^{(n+1)}\right) + \tau^2$.

GPR PREDICTION

Hence

$$p\left(y^{(n+1)} \middle| y^{(1)} = t^{(1)}, \ldots, y^{(n)} = t^{(n)}\right) \sim \mathcal{N}\left(\mu, \sigma^2\right),$$

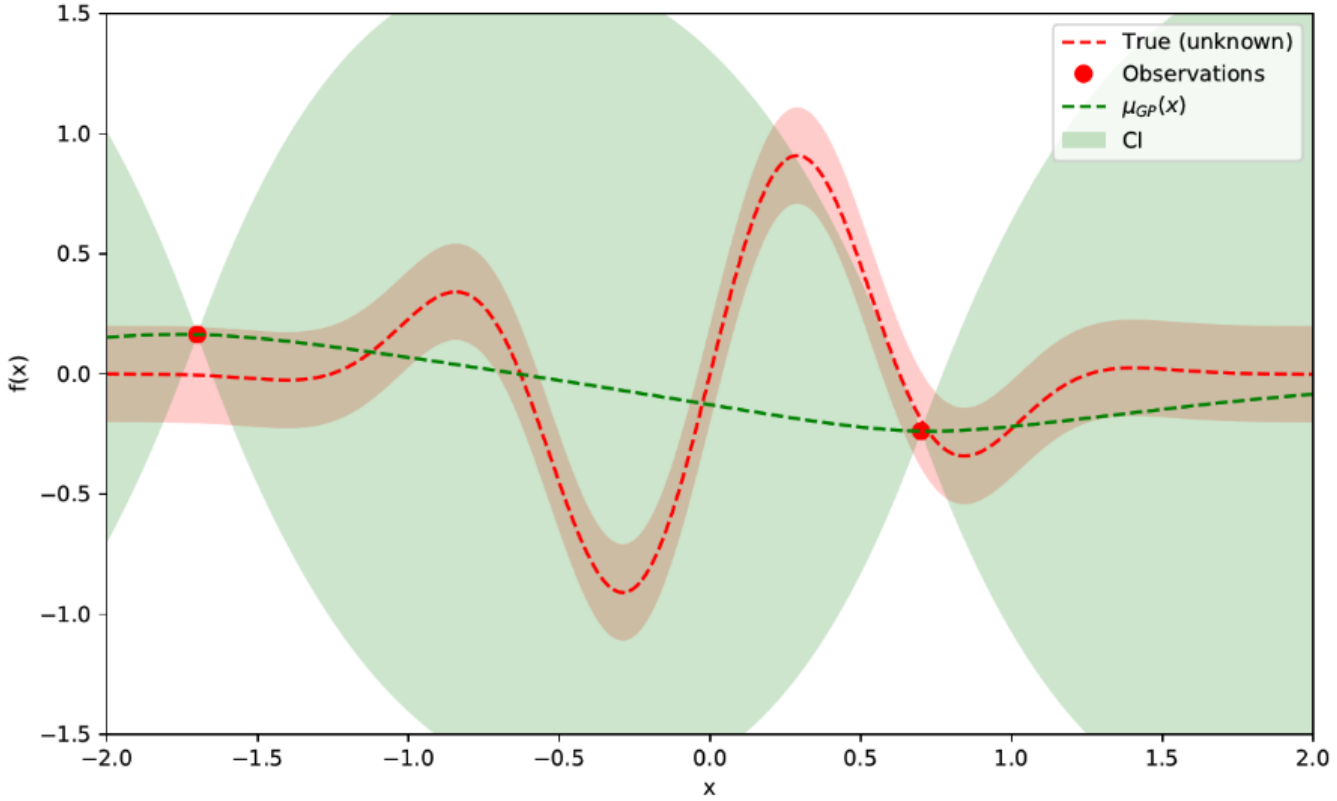
where

$$\mu = k^T \left(C^n\right)^{-1} \mathbf{t}_n,$$
$$\sigma^2 = c - k^T \left(C^n\right)^{-1} k,$$

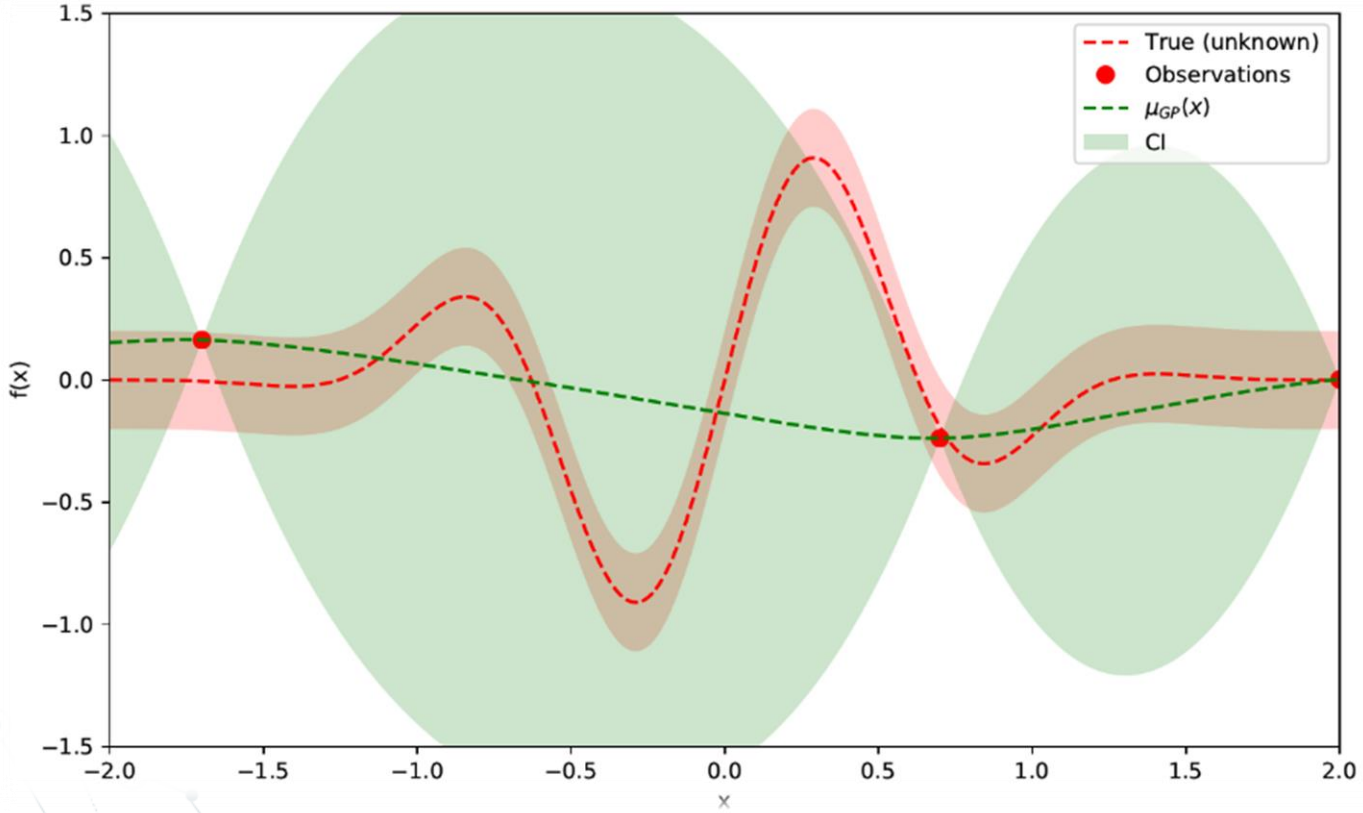
and we denote $\mathbf{t_n} = \left[t^{(1)}, \ldots, t^{(n)}\right]^T$.

Note that μ and σ can be viewed as functions of $x^{(n+1)}$.

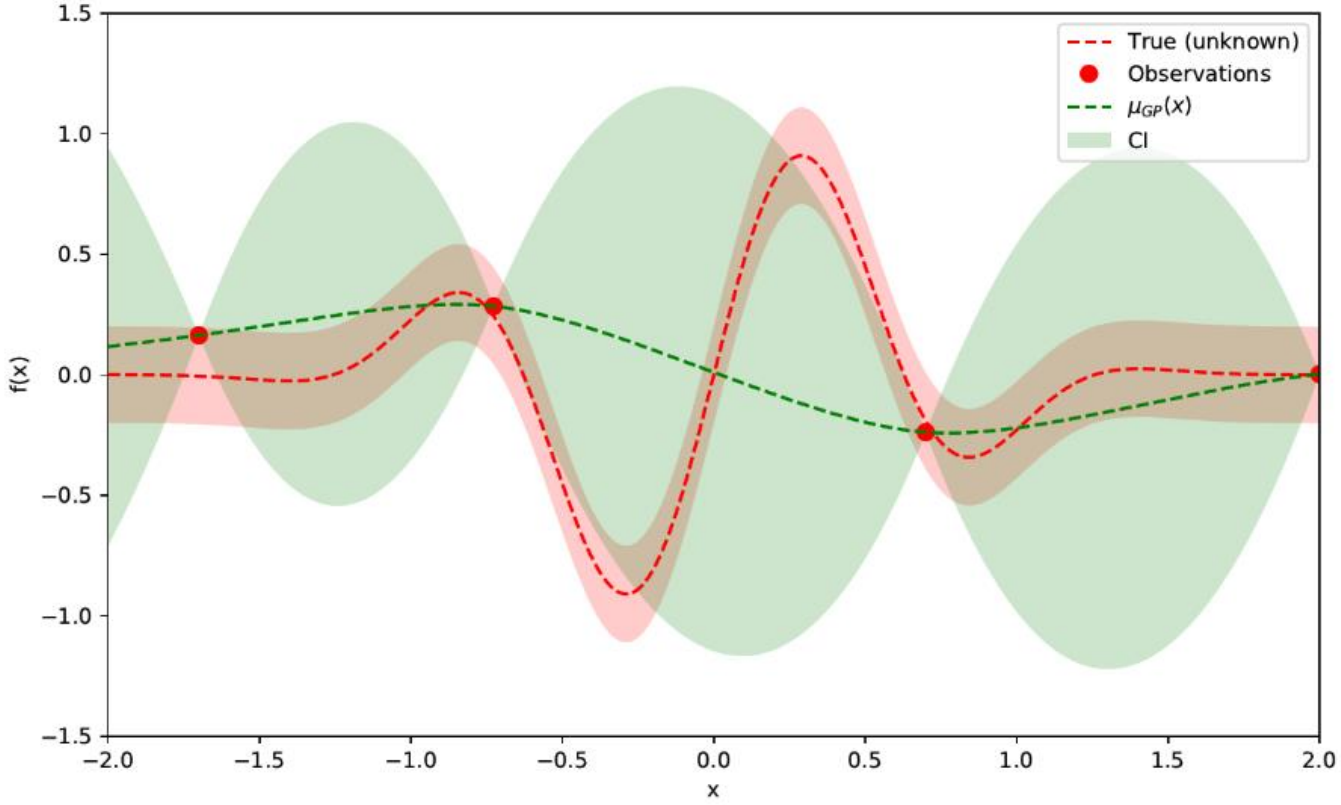
GPR PREDICTION



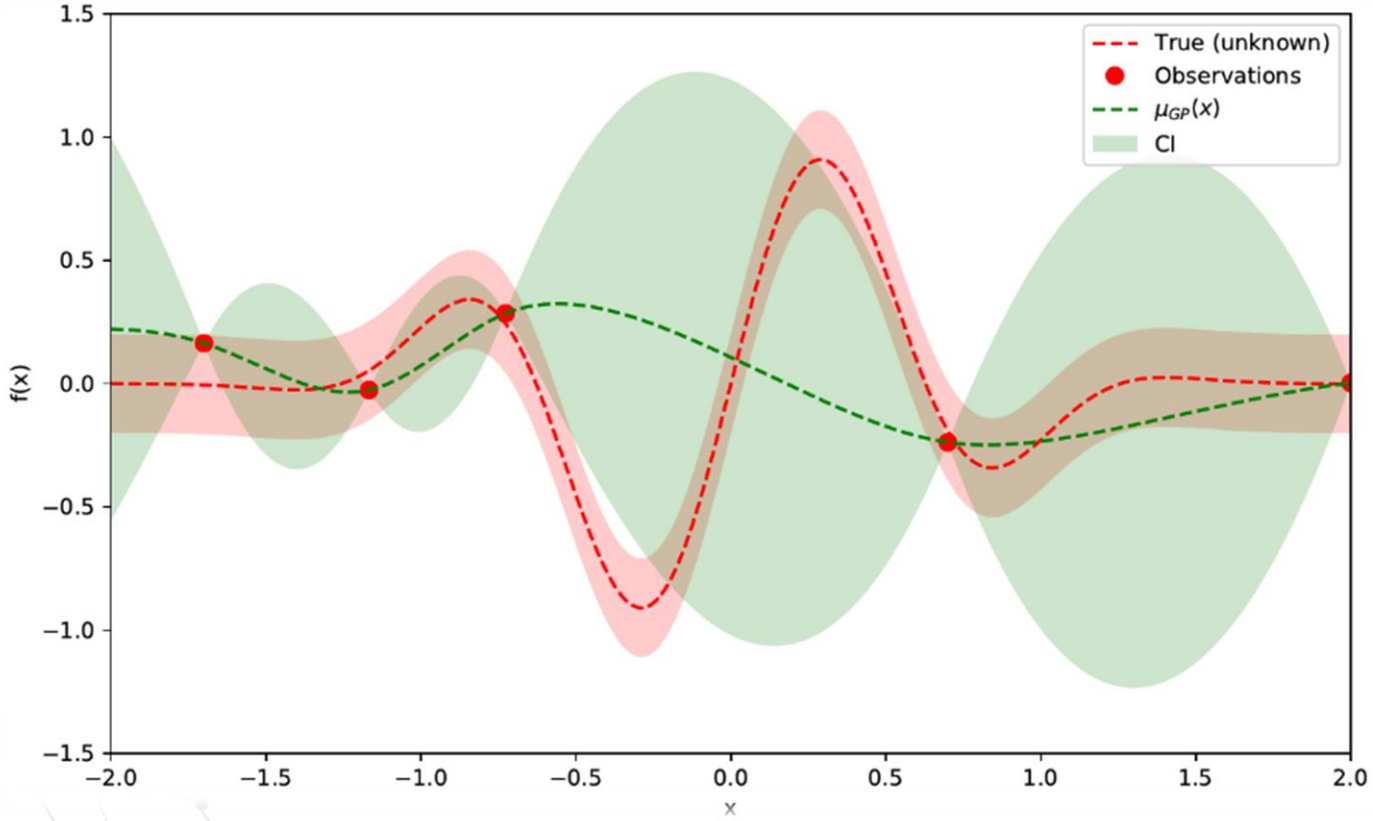
GPR PREDICTION



GPR PREDICTION



GPR PREDICTION



GPR PREDICTION

Given $x^{(1)}, \ldots, x^{(n+1)}$,

$$\text{prior distribution} \xrightarrow{\left\{t^{(i)}\right\}_{i=1}^{n}} \text{posterior distribution},$$

i.e.

$$p\left(y^{(1)}, \ldots, y^{(n+1)}\right) \sim \mathcal{N}\left(0, C^{n+1}\right) \longrightarrow p\left(y^{(n+1)} \,\middle|\, y^{(1)} = t^{(1)}, \ldots, y^{(n)} = t^{(n)}\right) \sim \mathcal{N}\left(\mu, \sigma^2\right).$$

Hence, the prior distribution is entirely determined by the kernel K, whereas the the posterior distribution is continually being updated by the observations $t^{(i)}$.

ADVANTAGES

- In many machine learning problems, the objective function f is a black-box function which does not have an analytic expression (i.e. one cannot take derivatives), or has one that is too costly to compute.
- Moreover, f may be expensive to evaluate, or its domain may be high-dimensional, which makes a grid-search over its domain prohibitively time-consuming (curse of dimensionality).

BAYESIAN OPTIMIZATION

- Bayesian optimization techniques attempt to find the global optimum of f in as few steps as possible.
- How?
  - Define a *surrogate model* which approximates the objective function f, eg. a Gaussian process.
  - Use an *acquisition function* to direct sampling to areas where one will have an increased probability of finding the optimum.

BAYESIAN OPTIMIZATION

In the start, select a kernel to model the objective function and choose an acquisition function $A(x)$. Now assume we are at time n where the n^{th} data-point $x^{(n)}$ and the corresponding value $y^{(n)}$ has just been sampled.

1. Update the posterior distribution with $y^{(n)}$.
2. Find the next sampling point $x^{(n+1)}$ by optimizing

$$x^{(n+1)} = \arg\max_x A\left(x \,\middle|\, x^{(1)}, \ldots, x^{(n)}\right)$$

3. Obtain a (possibly noisy) sample $y^{(n+1)} = f\left(x^{(n+1)}\right) + \epsilon^{(n+1)}$.

ACQUISITION FUNCTIONS

Let f^* denote the maximum value for f found so far, and let $\gamma_x = \frac{\mu_x - f^*}{\sigma_x}$.

1. Probability of Improvement:

$$A(x, f^*) = P(f_x > f^*) = \mathbf{\Phi}(\gamma_x),$$

2. Expected Improvement:

$$A(x, f^*) = \mathbb{E}[\max\{f_x - f^*, 0\}] = \sigma_x[\gamma_x \mathbf{\Phi}(\gamma_x) + \phi(\gamma_x)]$$

3. Upper Confidence Bound (UCB):

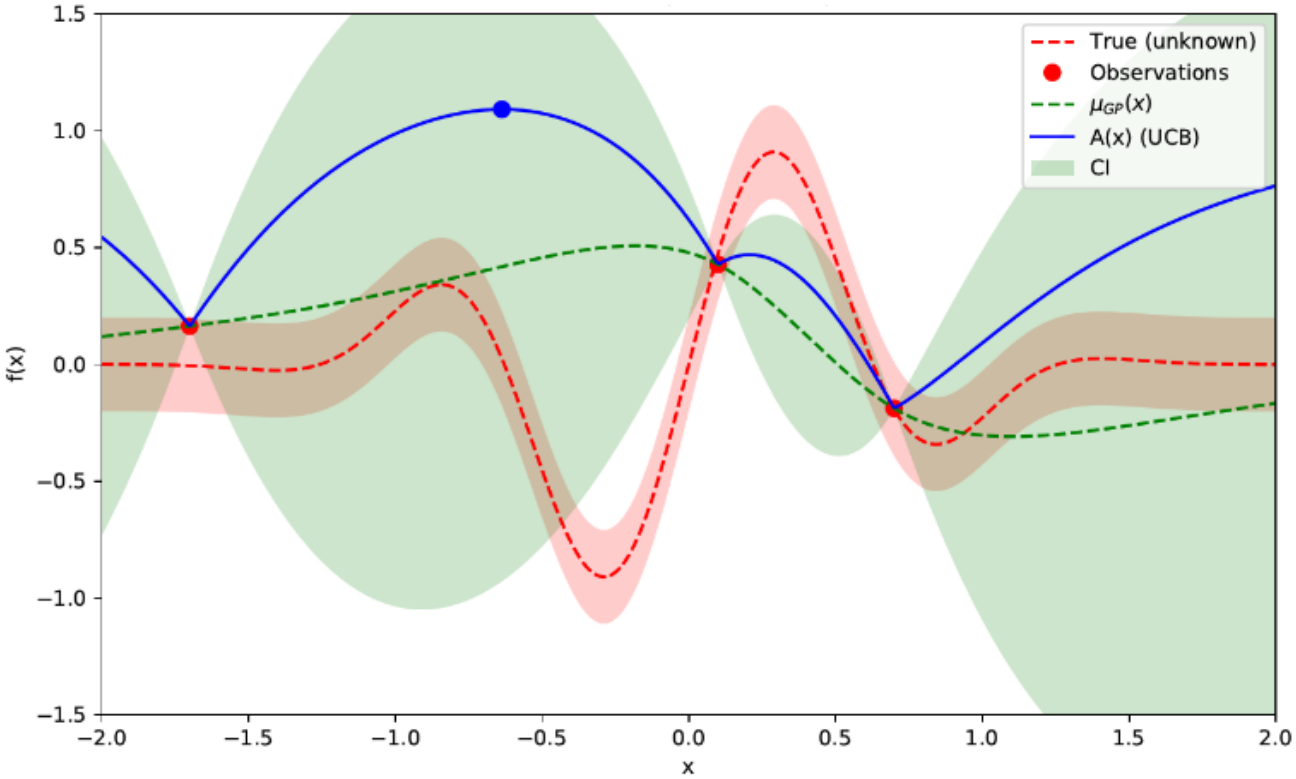
$$A(x) = \mu_x + \kappa\sigma_x$$

($\mathbf{\Phi}$ and ϕ denote the cdf and pdf of the standard normal distribution, and note that $f_x \sim \mathcal{N}(\mu_x, \sigma_x^2)$ denotes the predicted value of f at x)

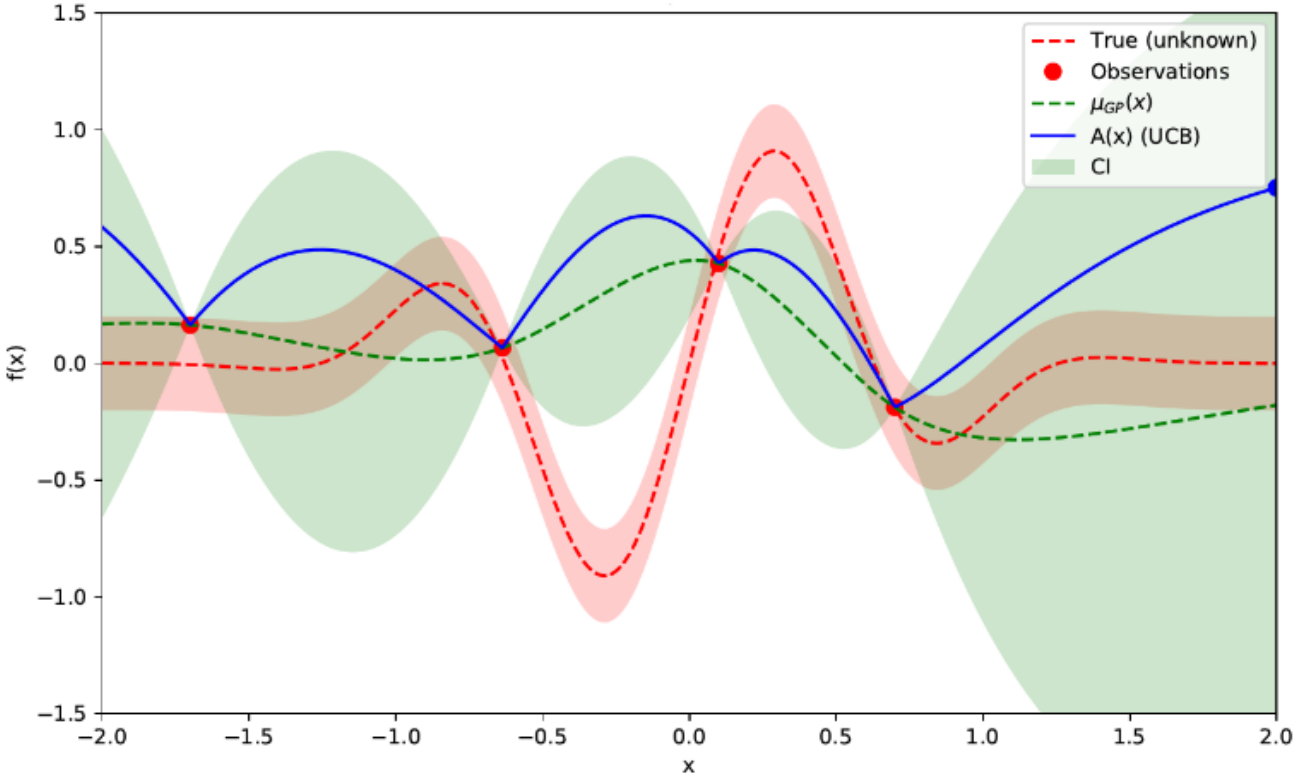
ACQUISITION FUNCTIONS

- The acquisition functions are relatively simple functions of μ_x and σ_x.
- Thus, Gaussian process regression replaces the original intractable objective function f with a tractable acquisition function which can be optimized by conventional methods, eg. gradient descent.

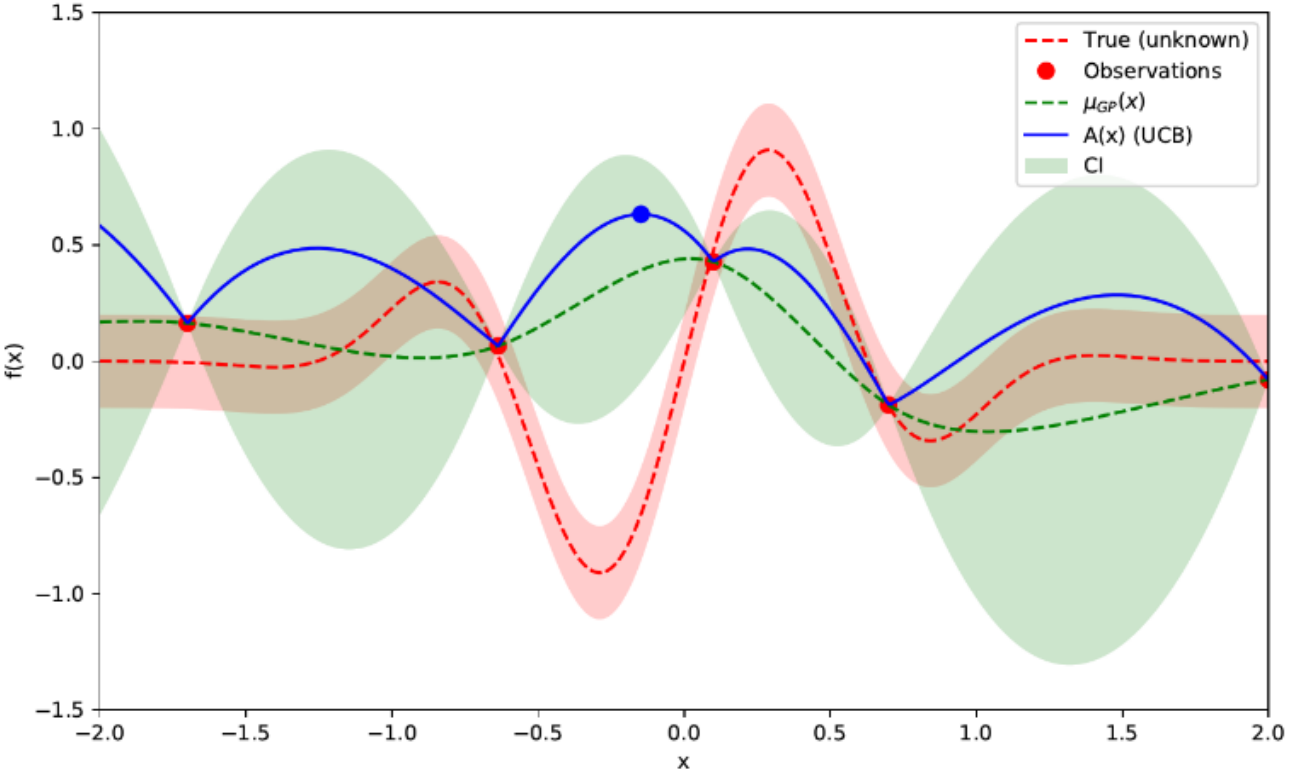
BAYESIAN OPTIMIZATION



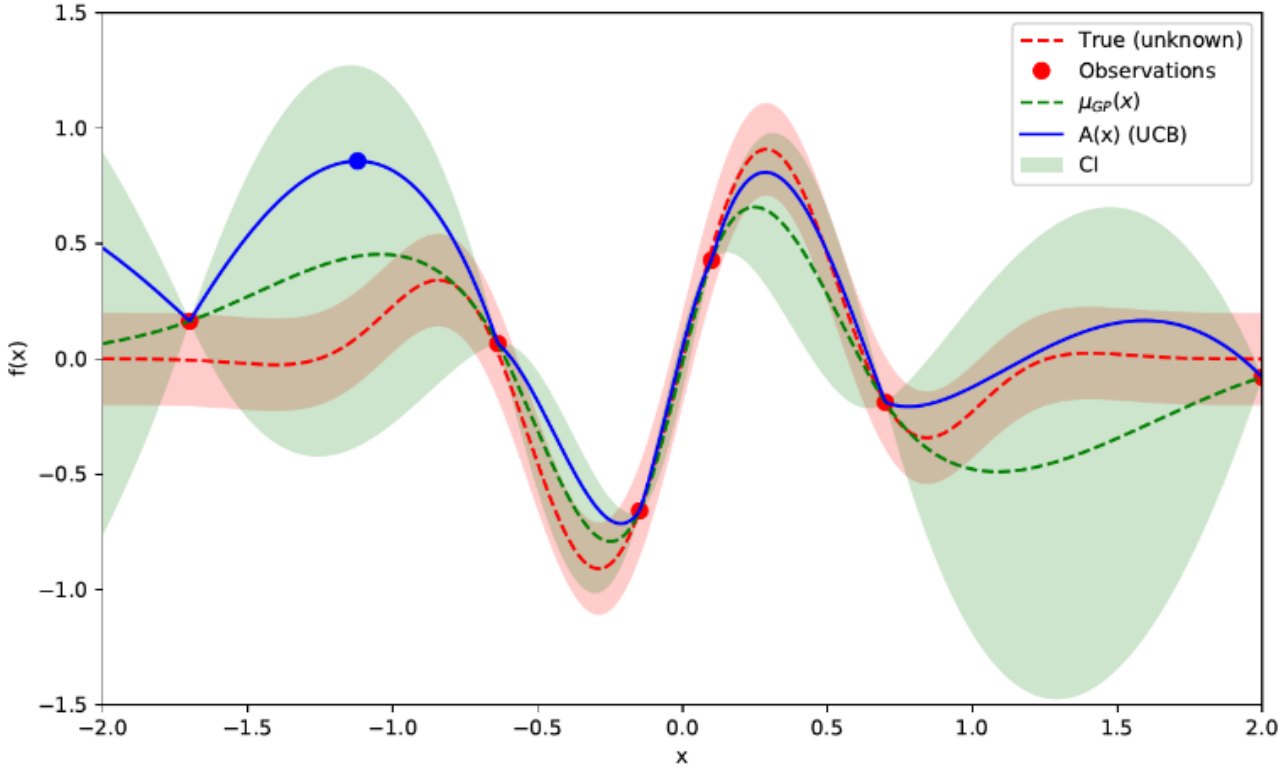
BAYESIAN OPTIMIZATION



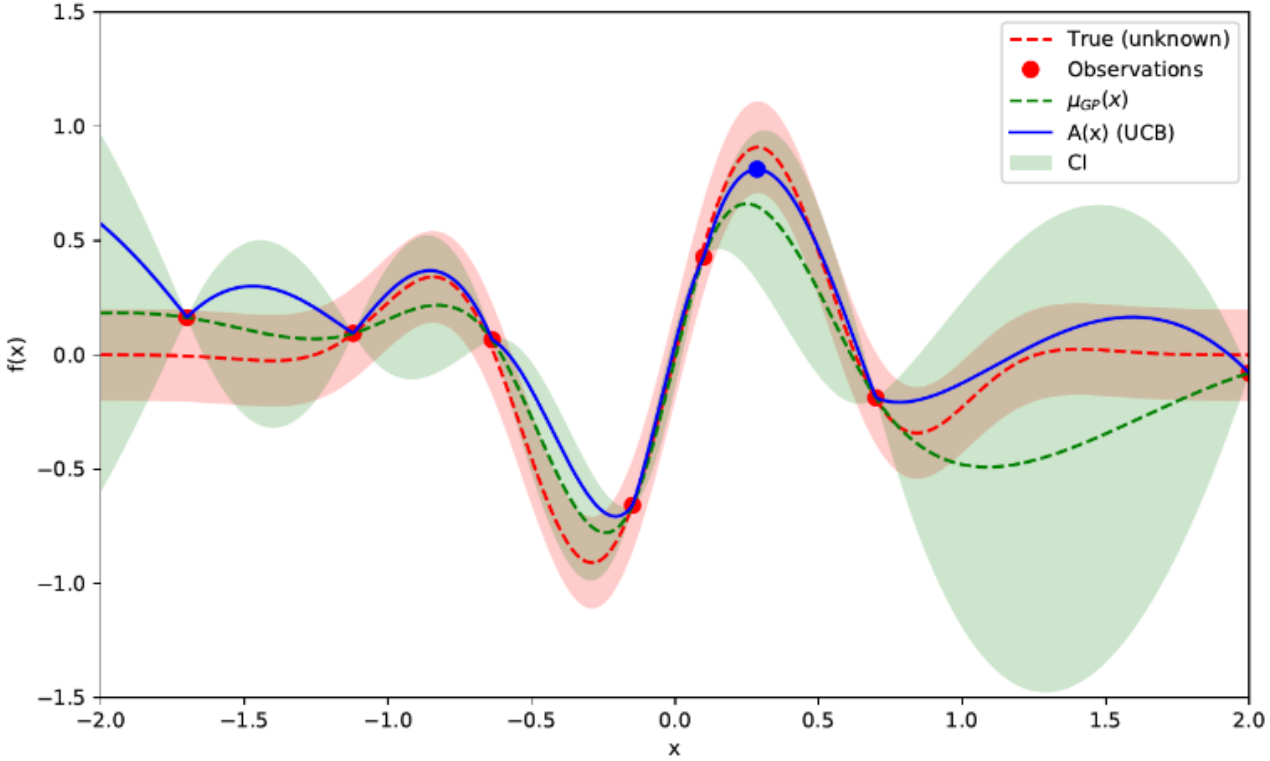
BAYESIAN OPTIMIZATION



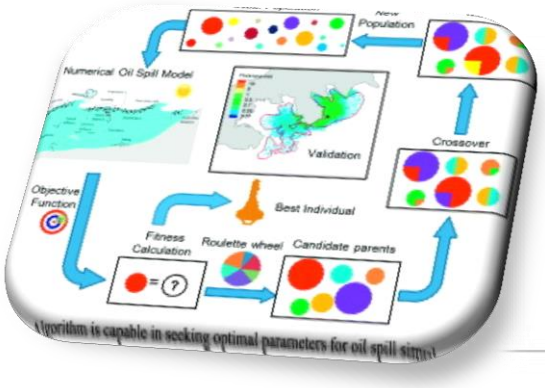
BAYESIAN OPTIMIZATION

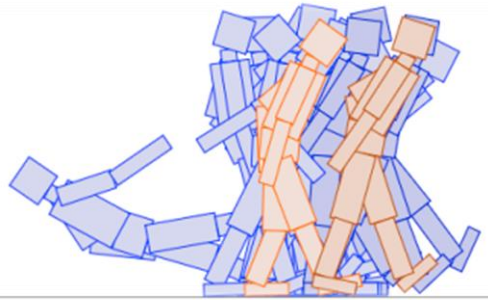


BAYESIAN OPTIMIZATION



GENETIC ALGORTIHMS





HISTORY

- Pioneered by John Holland in the 1970's
- Got popular in the late 1980's
- Based on ideas from Darwinian Evolution
- Can be used to solve a variety of problems that are not easy to solve using other techniques

BORROWING FROM GENETICS

- Each cell of a living thing contains ***chromosomes*** - strings of *DNA*
- Each chromosome contains a set of ***genes*** - blocks of DNA
- Each gene determines some aspect of the organism (like eye colour)
- A collection of genes is sometimes called a ***genotype***
- A collection of aspects (like eye colour) is sometimes called a ***phenotype***
- Reproduction involves recombination of genes from parents and then small amounts of ***mutation*** (errors) in copying
- The ***fitness*** of an organism is how much it can reproduce before it dies
- Evolution based on "survival of the fittest"

HYPOTHETICALLY…

- Suppose you have a problem
- You don't know how to solve it
- What can you do?
- Can you use a computer to somehow find a solution for you?
- This would be nice! Can it be done?

TRIVIAL SOLUTION

A "blind generate and test" algorithm:

Repeat

Generate a random possible solution

Test the solution and see how good it is

Until solution is good enough

IS IT EFFECTIVE?

- Sometimes - yes:
  - if there are only a few possible solutions
  - and you have enough time
  - then such a method *could* be used
- For most problems - no:
  - many possible solutions
  - with no time to try them all
  - so this method *can not* be used

A MORE EFFECTIVE METHOD

Generate a *set* of random solutions

Repeat

Test each solution in the set (rank them)

Remove some bad solutions from set

Duplicate some good solutions

make small changes to some of them

Until best solution is good enough

ENCODING DATA

How do you encode a solution?

- Obviously this depends on the problem!
- GA's *often* encode solutions as fixed length "bitstrings" (e.g. 101110, 111111, 000101)
- Each bit represents some aspect of the proposed solution to the problem
- For GA's to work, we need to be able to "test" any string and get a "score" indicating how "good" that solution is

ALGORITHM

For N generations:

1. Select parents using fitness function
2. Produce offsprings through recombination
3. Introduce mutations into offsprings
4. Replace parents with offsprings

After N generations, choose offspring with highest fitness function.